

# External Database Integration Design — v3.1

This is one of the project's core commercial moats: 90% of 50-100 person factories already use Dingxin / Chenghang / SuperBase / Excel. "Can you read my legacy data?" is the #1 ERP procurement killer question.

Without this capability, every customer demo dies. With it, a single customer is worth \$30k/year.

For full Chinese rationale and details see [EXTERNAL\\_DB\\_INTEGRATION\\_DESIGN\\_ZH.md](#).

## 1. Strategic Position

### 1.1 Why this matters more than mobile

| Customer says                | Real meaning                       | Without this we    |
|------------------------------|------------------------------------|--------------------|
| "We use Dingxin now"         | Won't replace it, you must read it | demo dies          |
| "I have 500k legacy records" | Won't re-key, need migration       | can't sign         |
| "Connect to our SQL Server"  | ODBC required                      | integrators block  |
| "Customer PO comes in Excel" | Must ingest CSV / Excel            | sales has no story |

Migration anxiety is the top procurement killer. With external DB integration: "Your Dingxin doesn't need to die. We read it; AI auto-maps fields; you click ConfirmCard; done."

### 1.2 Same DNA as Conversational ERP

This is not a side track — it directly extends the "natural-language replaces training" promise:

"Director Wang types: 'How much in Dingxin orders for May?' → AI cross-DB query"

"Procurement types: 'Migrate Dingxin customers' → AI emits schema-mapping ConfirmCard"

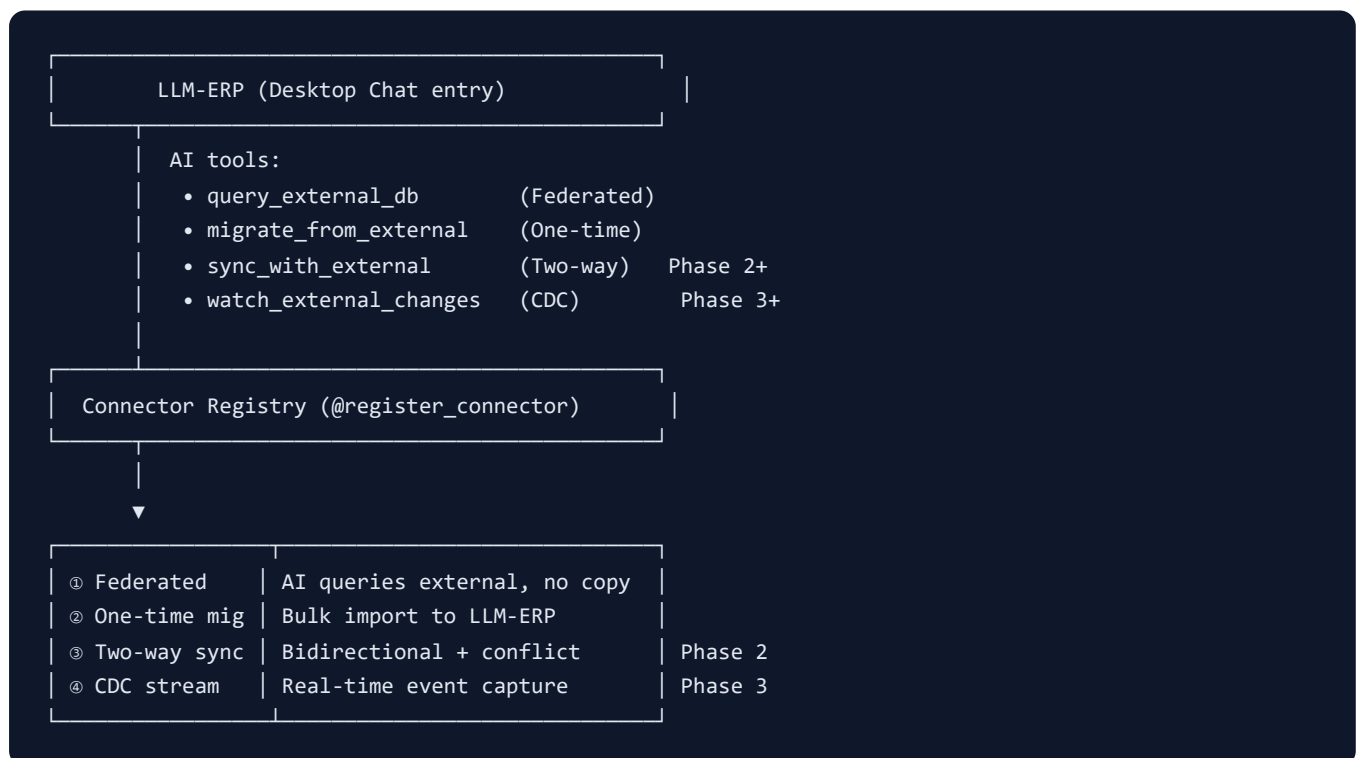
Tool registry / RiskTier / ConfirmCard from Phase 1 are all reused.

## 2. Legacy Systems Coverage

| Customer's system       | Connection                  | Priority | Difficulty |
|-------------------------|-----------------------------|----------|------------|
| Dingxin Workflow ERP    | SQL Server (pyodbc)         | 🔥 P0     | ★ ★        |
| Chenghang ERP           | SQL Server                  | 🔥 P0     | ★ ★        |
| SuperBase ERP           | REST API (OAuth2)           | 🟡 P1     | ★ ★        |
| SAP Business One        | DI API / Service Layer REST | 🟡 P1     | ★ ★ ★      |
| Odoo                    | XML-RPC / REST              | 🟡 P1     | ★ ★        |
| In-house Access / Excel | ODBC / openpyxl             | 🟢 PoC    | ★          |
| CSV batches             | Folder watcher              | 🟢 PoC    | ★          |
| e-invoice SaaS          | REST API                    | 🟡 P1     | ★ ★        |
| PLC / SCADA / MES       | OPC UA / Modbus             | 🛑 P5+    | ★ ★ ★ ★ ★  |

MVP target: PoC = sqlite + csv. Phase 1.5 = + SqlServer + REST API.

## 3. Four Connection Modes



v3.0 scope: ① + ②. ③ ④ deferred pending customer feedback.

## 4. Architecture

### 4.1 Code structure

```
backend/app/integrations/connectors/  
├─ __init__.py  
├─ base.py           # Connector ABC + ConnectorMeta  
├─ registry.py       # @register_connector / get_connector  
├─ exceptions.py     # ConnectorError / TableNotFound / ...  
├─ sqlite_connector.py # ✅ built-in (PoC)  
├─ csv_connector.py   # ✅ built-in (PoC)  
├─ postgres_connector.py # Phase 1.5  
├─ sqlserver_connector.py # Phase 1.5 (Dingxin / Chenghang)  
├─ mysql_connector.py  # Phase 1.5  
├─ rest_api_connector.py # Phase 1.5  
└─ excel_connector.py  # Phase 1.5
```

### 4.2 Connector interface contract

```
class Connector(ABC):  
    meta: ConnectorMeta  
  
    def __init__(self, config: dict): ...  
  
    @abstractmethod  
    async def test_connection(self) -> bool: ...  
  
    @abstractmethod  
    async def list_tables(self) -> list[str]: ...  
  
    @abstractmethod  
    async def query(  
        self, table: str,  
        filters: dict | None = None,  
        limit: int = 100,  
    ) -> list[dict]: ...  
  
    async def schema_of(self, table: str) -> list[dict]: ...
```

## 4.3 Four security defenses

| Layer | Defense               | Why                                                                  |
|-------|-----------------------|----------------------------------------------------------------------|
| 1     | read-only by default  | Never accidentally write to legacy system                            |
| 2     | table whitelist       | <code>query(table)</code> must pass <code>list_tables()</code> check |
| 3     | parameterized filters | No SQL string concatenation, prepared statements only                |
| 4     | audit log             | Every external query logged with connection / table / filter         |

## 5. AI Tool Specs

### 5.1 PoC phase (v3.0) — 3 read tools

| Tool                                   | RiskTier | Description                             |
|----------------------------------------|----------|-----------------------------------------|
| <code>list_external_connections</code> | READ     | List configured external DB connections |
| <code>list_external_tables</code>      | READ     | List tables in a connection             |
| <code>query_external_db</code>         | READ     | Cross-DB query with filters + limit     |

### 5.2 Phase 1.5 — add 4 more

- `register_external_connection_with_confirm` (HARD\_WRITE)
- `delete_external_connection_with_confirm` (HARD\_WRITE)
- `preview_schema_mapping` (READ — AI auto-suggests field mappings)
- `migrate_from_external_with_confirm` (HARD\_WRITE — one-time bulk import)

## 6. Killer Conversational Scenarios

### Scenario A: Federated query

```
Wang: "How much in Dingxin orders for May?"
AI: list_external_connections → finds legacy_dingxin
    list_external_tables(legacy_dingxin) → finds OrderHeader
    query_external_db(legacy_dingxin, OrderHeader,
                      filters={order_date_gte: 2026-05-01})
    Sums Amount → "$3.2M (45 orders) in Dingxin; $580K (12 orders) in LLM-ERP."
```

### Scenario B: One-time migration

```
Alice: "Migrate customers from Dingxin"
AI: list_external_tables → Customer
    preview_schema_mapping → auto suggests:
      Dingxin.Customer.CustNo → LLM-ERP.Customer.code
      Dingxin.Customer.CustName → LLM-ERP.Customer.name
      Dingxin.Customer.Grade → LLM-ERP.Customer.grade
    Emits ConfirmCard: "Import 124 customers. Conflict policy: overwrite by code."
    Alice clicks Confirm → migration runs → progress bar → done.
```

### Scenario C: CSV folder watch

```
Sales: "Customer A drops PO CSVs in D:/orders daily at 9am"
AI: register_external_connection(name=customer_a_csv, connector=csv_folder,
                                config={folder: D:/orders})
    schedule_external_sync(every=5min, mode=ingest_new_files,
                           target=sales_order)
    "Set up. Every 5 minutes scans D:/orders for new CSVs and imports as SO."
```

## 7. Schema Mapping AI

### 7.1 Auto-suggestion logic

1. Column name match (exact + LLM fuzzy via Glossary)
2. Type inference ( `VARCHAR(50)` ↔ String, `DECIMAL(18,2)` ↔ Float)
3. Domain constraints (LLM-ERP `customer.grade` ∈ {A,B,C,D} → warn)
4. Confidence score:
  - 95%+ → auto apply
  - 70-95% → emit ConfirmCard for human review
  - <70% → ask "no match found, create new field?"

## 7.2 Reuses Phase 2 Glossary

The Glossary from `CONVERSATIONAL_ERP_DESIGN_EN.md` directly applies:

- "Customer" / "CustNo" / "Cust" → all map to LLM-ERP.Customer
- "Part" / "Material" / "Item" → all map to LLM-ERP.Part

## 8. Phase Plan

### Phase 1.5 (parallel with Phase 1, 2 weeks)

| #      | Task                                            | Days | GAP   |
|--------|-------------------------------------------------|------|-------|
| 1.5.1  | Connector framework (base + registry)           | 0.5  | G-501 |
| 1.5.2  | SqliteConnector + CsvFolderConnector (PoC)      | 0.5  | G-502 |
| 1.5.3  | 3 read tools (list / list_tables / query)       | 0.5  | G-503 |
| 1.5.4  | Smoke tests (PoC)                               | 0.5  | G-504 |
| 1.5.5  | SqlServerConnector (pyodbc)                     | 1    | G-505 |
| 1.5.6  | Postgres + MySQL connectors                     | 1    | G-506 |
| 1.5.7  | RestApi connector + SuperBase / SAP B1 profiles | 2    | G-507 |
| 1.5.8  | Schema mapping AI                               | 2    | G-508 |
| 1.5.9  | migrate_from_external_with_confirm              | 1.5  | G-509 |
| 1.5.10 | Customer system profiles (Dingxin / Chenghang)  | 1    | G-510 |

**Total:** ~10.5 work days, parallel with Phase 1.

## 9. Risks & Mitigations

| Risk                                    | Impact                   | Mitigation                                                  |
|-----------------------------------------|--------------------------|-------------------------------------------------------------|
| Driver install pain (SQL Server pyodbc) | IT frustration           | Pre-installed in Docker; <code>install.sh</code> one-click  |
| Legacy schemas vary wildly              | Mapping AI errors        | Pre-built profiles for common systems + human final confirm |
| Accidental writes to legacy             | Customer data corruption | read-only default; hard-write requires ConfirmCard          |
| SQL injection                           | Security incident        | Table whitelist + parameterized statements only             |
| Slow federated queries                  | Bad UX                   | Default limit 100; big queries → background job + Email     |

## 10. Sales Story

### One-liner

"Your Dingxin doesn't have to die. We read it. AI migrates gradually. Zero risk during transition."

### Three-liner

"ERP buyers' #1 fear is legacy data. We connect directly to Dingxin / Chenghang / SuperBase / SAP B1 / Excel / CSV. AI auto-maps fields; you click a ConfirmCard; done — no SQL required. 90% of competitors want you to 'kill the old system first.' We say 'run both for 6 months if you want.' **Zero-risk transition.**"

**Last updated:** 2026-05-15 (v3.1 strategic supplement: External DB integration) **Related GAPS:** G-501 ~ G-510 **Roadmap:** Phase 1.5